

Concurrency Control

- ⊗ Concurrency control
- ⊗ Lock base protocol
- ⊗ Compatibility matrix
- ⊗ Two phase locking protocol
- ⊗ Time base protocol
- ⊗ Graph base protocol
- ⊗ Dead lock
- ⊗ Checkpoint
- ⊗ Distributed database system
- ⊗ Distributed vs centralized
- ⊗ Replication
- ⊗ Fragmentation
- ⊗ Diffⁿ Architecture

Concurrency control: Concurrency control is a very important concept of DBMS which ensures the simultaneous execution or manipulation of data by several processes or users without resulting in data inconsistency.

Concurrency control provides a procedure that is able to control concurrent execution of the operations in the database.

Advantages:

- ① Waiting time will be decreased
- ② Response time will decrease
- ③ Resource utilization will increase
- ④ System performance and efficiency is increased.

Lock based protocols: while one transaction is accessing a data item, no other transaction can modify that data item.

A lock is a mechanism to control concurrent access to a data item.

Data items can be locked in two modes:

1. Shared: Denoted by S. Data item can only be read but cannot write

Role of compatibility matrix in lock based protocol

1. A transaction may be granted a lock on item if the requested lock is compatible with the locks already held on the item by other transactions.

	S	X
S	true	false
X	false	false

2. Any number of transactions can hold shared locks on an item but if any transaction hold an exclusive lock on the item, no other transaction may hold any lock on them.
3. If a lock can't be granted, the requesting transaction is made to wait till all ⁱⁿ⁻compatible locks held by other transactions have been ~~ready~~ released.
4. A transaction can unlock a data item $\&$ by the `unlock(&)` instruction. Transaction T_i may unlock a data item that it held locked at some earlier point.

Pitfalls of lock-based protocol

a. Consider the partial schedule of fig. 1 for transaction T_3 and T_4

T_3	T_4
lock-x(B)	
read(B)	
$B := B - 50$	
write(B)	
	lock-s(A)
	read(A)
	lock-s(B)
lock-x(A)	

b. Neither of the transactions can make progress

c. Such a situation is called deadlock.

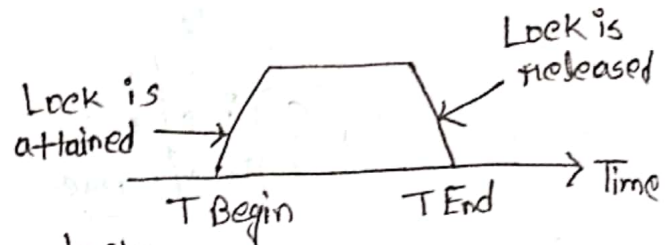
Working difference between two phase locking protocol and time stamp based protocol:

Two phase locking protocol: The protocol ensure conflicted serializable schedule.

Phase-1 : Growing phase

⇒ Transaction may obtain locks

⇒ Transaction may not release locks



Phase-2 Shrinking phase

⇒ Transaction may release lock

⇒ Transaction may not obtain locks.

Types: 1. Strict two phase locking protocol
(Requires Exclusive mode locks taken)

2. Rigorous two phase locking
(Requires all locks)

Time based protocol: Each transaction is issued with a timestamp when it enters the system.

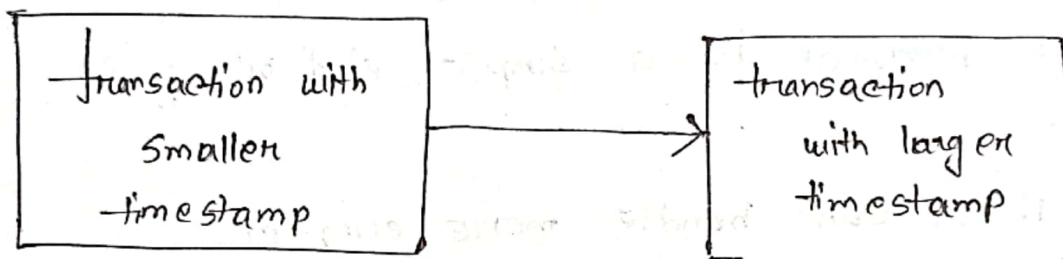
If an old transaction T_i has time-stamp $TS(T_i)$,
a new transaction T_j has time stamp $TS(T_j)$
such that, $TS(T_i) < TS(T_j)$

Two time-stamp Values:

W-time stamp: W-time-stamp(Q) is the largest timestamp of any transaction that executed write(Q) successfully.

R-time stamp: R-time stamp(Q) is the largest timestamp of any transaction that executed read(Q) successfully.

Connection of timestamp ordering protocol:



Methodology of time stamp based protocol:

It works as-

- ① If a transaction T_i issues a read(x) operation:
 - a. If $TS(T_i) < W\text{-time stamp}(Q)$ then T_i needs to read a value of Q and that was already overwritten. Read operation is rejected and T_i is rolled back.
 - b. If $TS(T_i) \geq W\text{-time stamp}(Q)$, then the read operation is executed and $R\text{-timestamp}(Q)$ is set to the maximum of $R\text{-timestamp}(Q)$ and $TS(T_i)$.
- ② If a transaction T_i issues a write(x) operation:
 - a. If $TS(T_i) < R\text{-time stamp}$ then the value of Q that T_i is producing was needed previously.
 - b. If $TS(T_i) < W\text{-time stamp}(Q)$. Then T_i is attempting

to write and value of Q .

The system rejects the write operation and rolls T_i back.

c. Otherwise the system executes the write operation and sets w -time stamp (Q) to $Ts(T_i)$.

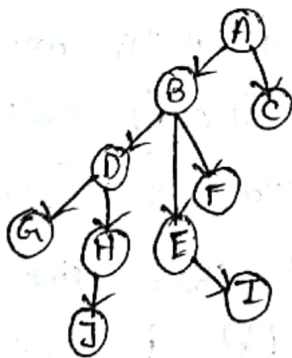
Graph-base protocol: when a transaction starts, a node is created in the graph representing the transaction.

⇒ The tree protocol is a simple kind of graph protocol.

Advantage: 1. It can handle more complex transaction dependencies than other protocols such as two-phase locking.

Disadvantage: Expensive

Tree protocol:



⇒ It ensures serializability

⇒ Increase in concurrency

⇒ Protocol is dead-lock free
no rollback are required

⇒ Data items may be
unlocked at any time.

Dead-lock

A system is in a deadlock state if there exists a set of transactions such that every transaction in the set is waiting for another transaction in the set.

Prevention strategies of deadlock:

① wait-die scheme: This is a non preemptive technique. When transaction T_i requests a data item currently held by T_j , T_i is allowed to wait only if T_i has a smaller timestamp than that of T_j . Otherwise T_i is rolled back (dies).

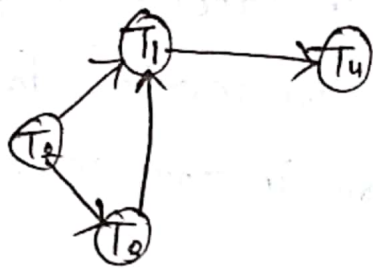
Example: Suppose that transaction T_{22} , T_{23} , T_{24} have timestamp 5, 10, 15 respectively. If T_{22} requests a data item held by T_{23} , then T_{22} will wait. If T_{24} request a data item held by T_{23} , T_{24} will be rolled back.

② wound-wait scheme: This is a preemptive technique.

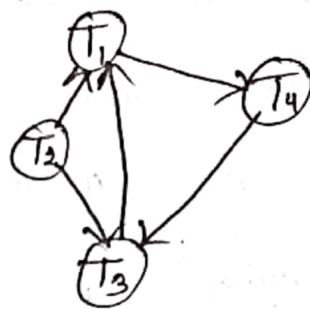
It is a counter part to the wait-die scheme. When transaction T_i requests a data item held by T_j it is allowed to wait only if T_i has longer timestamp than T_j . Otherwise T_j is rolled back.

Example: If T_{22} requests a data item held by T_{23} , T_{23} will be rolled back. If T_{24} requests a data item held by T_{23} , T_{24} will wait.

Deadlock Detection:



wait-for graph
without a cycle



wait-for graph
with a cycle

Checkpoint: The checkpoint is used to declare a point before which the DBMS was in a consistent state and all transactions were committed.

Distributed database: A distributed database is basically a database that is not limited to one system, it is spread over different sites. It is located on various sites that don't share physical components. DNS (Domain name service) is a well known as distributed database.

Centralized database: A centralized database manages the database as if it were stored on the same computer.

Distributed database Vs centralized database:

Distributed database	Centralized database
① It contains two or more database files located at different locations in the network.	① It contains a single database located at one location in the network.
② If one database fails, the users can still access other database files.	② If the database fails, the users do not have access to a database.
③ Various databases is required	③ It is easier to update and manage data
④ More reliable	④ Less reliable
⑤ Database is maintained at a number of different sites	⑤ Data is maintained at one site

Distributed Database system:

1. Homogeneous Database: In a homogeneous database, all different sites store database identically.

⇒ The operating system, database management system, and data structures used - all are the same at all sites.

⇒ They are easy to manage.

2. Heterogeneous Database: In a heterogeneous distributed database, different sites can use different schema and software that can lead to problems in query processing and transactions.

⇒ They may use different data models for the database.

⇒ Transactions are required for different sites to communicate.

Distributed Database Storage: There are 2 ways in database where data can be stored on different sites.

1. Replication: Replication is the process of copying and maintaining database objects in multiple database

⇒ It increases the availability of data at different sites
(Advantage)

⇒ Disadvantage: Data needs to be constantly updated.

Fragmentation: Fragmentation is the system of partitioning the relation into several fragments and storing each fragment at a different site.

⇒ It is stored in different sites where they are required.

⇒ Advantage: It doesn't create ~~of~~ copies of data, consistency is not a problem.

Fragmentation of relations can be done in two ways:

1. Horizontal fragmentation: ~~The~~ π

⇒ Splitting by rows

⇒ The relation is fragmented into groups of tuples so that each tuple is assigned to at least one fragment.

2. Vertical fragmentation:

⇒ Splitting by columns

⇒ the schema of the relation is divided into smaller schemas.

⇒ Each fragment must contain a common candidate key so as to ensure a lossless join.

Advantage of distributed database system:

- ⇒ Fast data processing
- ⇒ Reliability and availability of this system is high
- ⇒ It is easier to expand the system
- ⇒ It possess reduced operating cost.
- ⇒ It has improved sharing ability and local autonomy.

Disadvantage of distributed system:

- ⇒ overload problem
- ⇒ It is difficult to manage.
- ⇒ Security issues often arise in many software and hardware

Different architectures for distributed database systems:

- ① Client-server architecture: ⇒ client connect to a central server, which manages the distributed system.
⇒ The server is responsible for coordinating transactions, managing data storage
- ② peer-to-peer architecture: Each site in the distributed database system is connected to all other sites
- ③ Federated architecture: Each site in the distributed database system maintain its own independent database.